

Agent Evaluations

Created by Diogo, Andre, last updated on May 20, 2026 • 12 minute read

- Overview
- Concepts
- User Flow
 - 1. Scenario Generation
 - 2. Evaluation History (without any previous evaluations)
 - 3. Create Evaluation
 - 4. Create/Edit Scenario
 - 5. Evaluation History (with existing evaluations)
 - 6. Evaluation Results
 - 7. Scenario Code View
- Design Contact
- Linked Items
- Resources

Design Contact

- @ Diogo, Andre
- @ Wong, Matthew

Linked Items

- BTPUXAI-118 - Getting issue details...
STATUS
- BTPUXAI-116 - Getting issue details...
STATUS

Overview

Offline Agent Evaluations is the quality assurance pathway within SAP Build's AI agent development tooling. Rather than waiting until an agent is in production to discover failures, evaluations allow business users and developers to define expected agent behavior upfront — through structured test cases called **scenarios** — and validate that behavior systematically across runs.

Evaluation scenarios are generated with the agent through the IBD flow and are available in the Design tab, inside each agent artifact's solution. The flow spans two phases: **Design Time** (where scenarios are authored and configured) and **Run Time** (where evaluations are executed and results are reviewed) — each producing progressively more precise evidence of agent quality. Due to current technical limitations for Sapphire, evaluations can only be run in post-deployment.

⚠ Due to some technical limitations for Sapphire timing, evals can only be run after the agent being deployed. However, the scenarios are still generated in the IBD flow.

Resources

- Design (Figma)
- User Flow (FigJam)

Concepts

Step	Artifact Produced	Purpose
1. Scenarios	Pre-generated test scenarios	Joule produces an initial set of scenarios during agent design time, grounded in the agent's known tools and intended use cases. These form the baseline test suite before any evaluation is run.
3. Evaluation	Configured evaluation ready to run	The user reviews the available scenarios, adds or removes test cases, and triggers the evaluation run. Scenarios can be generated on demand via Joule at this point.
4. Eval Runs	Evaluation history and performance trend	Once evaluations are run, results are recorded in a table alongside a regression/progression chart. Evaluations have states: Draft, In Progress Complete, and Canceled. Only one draft can exist at a time.
5. Eval Results	Per-scenario results and KPI metrics	Clicking a completed evaluation reveals the full results view: a KPI card summary and a breakdown of pass/fail status per scenario, covering tool validation, response quality, and rule compliance.

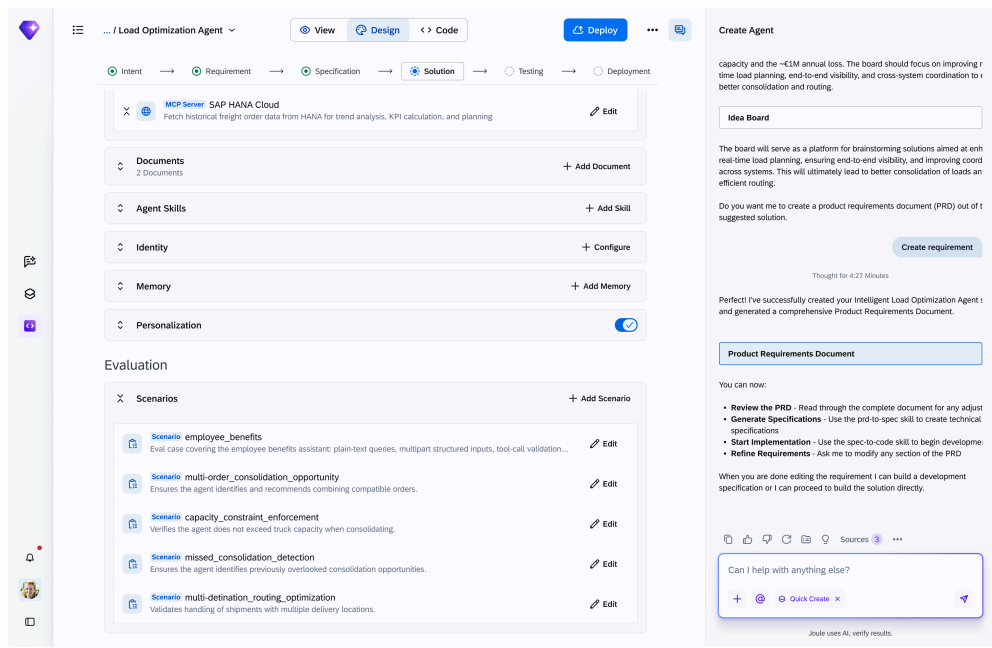
📘 **Design principle:** Each evaluation phase gates the next. Scenarios must be defined before an evaluation can be run, and results are only surfaced once an evaluation is complete. This prevents underspecified test cases from producing misleading quality signals.

User Flow

The sections below cover each screen state in sequence. Each includes the entry point, annotated elements, and relevant design notes.

1. Scenario Generation

Trigger: The user has completed the IBD flow and the agent artifact has been created. Joule automatically generates evaluation scenarios based on the agent's defined tools and capabilities. These are accessible from the **Design** tab within the agent artifact, before deployment.



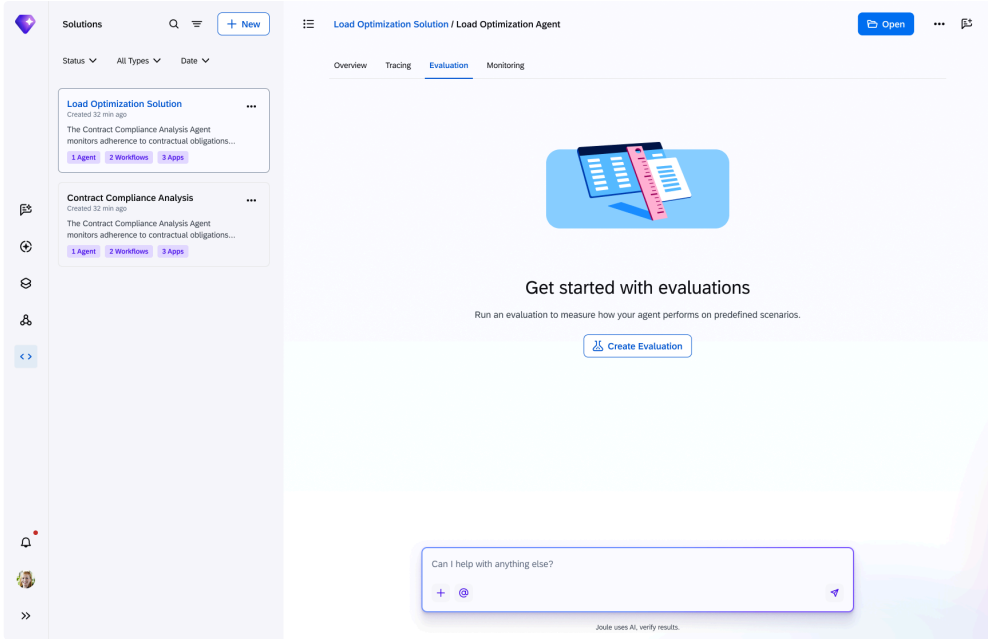
Scenarios are generated at design time, tied to the agent's current tool set. The Design tab presents the agent's full configuration alongside the scenario list, giving the user a single place to review both what the agent can do and how it will be tested.

Element	Behavior	Design Notes
Agent Settings panel	Collapsed section exposing model selection (combobox) and creativity/temperature (slider).	These settings govern how the agent behaves during evaluation runs. They can be adjusted before running without modifying the agent's deployed configuration.
Tools panel	Lists the tool groups and individual tools available to the agent, organized by source project. Each tool shows its name and a short description.	Tool visibility here reinforces the connection between agent capabilities and the scenarios that test them. Scenarios are grounded in these tools — adding or removing tools may affect scenario relevance.
Evaluation Scenarios panel	Pre-generated scenarios tied to the agent's known tools and context. Each entry displays its identifier and a short description of what it covers.	Scenario generation is automatic and non-blocking. The user doesn't need to author scenarios from scratch — the IBD flow produces a working baseline. Manual editing and Joule-assisted refinement are available as follow-on actions.
Joule conversation panel	Persistent right-side panel. Provides contextual guidance and allows the user to prompt Joule to refine scenarios or adjust configuration.	Carries full IBD context forward. The user can prompt Joule mid-flow without losing their place in the design view.

Leads to: The user deploys the agent. After deployment, evaluation runs become available from the **Evaluations** tab in runtime.

2. Evaluation History (without any previous evaluations)

Trigger: The agent has been deployed. The user navigates to the **Evaluations** tab within the agent artifact in runtime. No evaluations have been run yet, so the system displays an empty state.



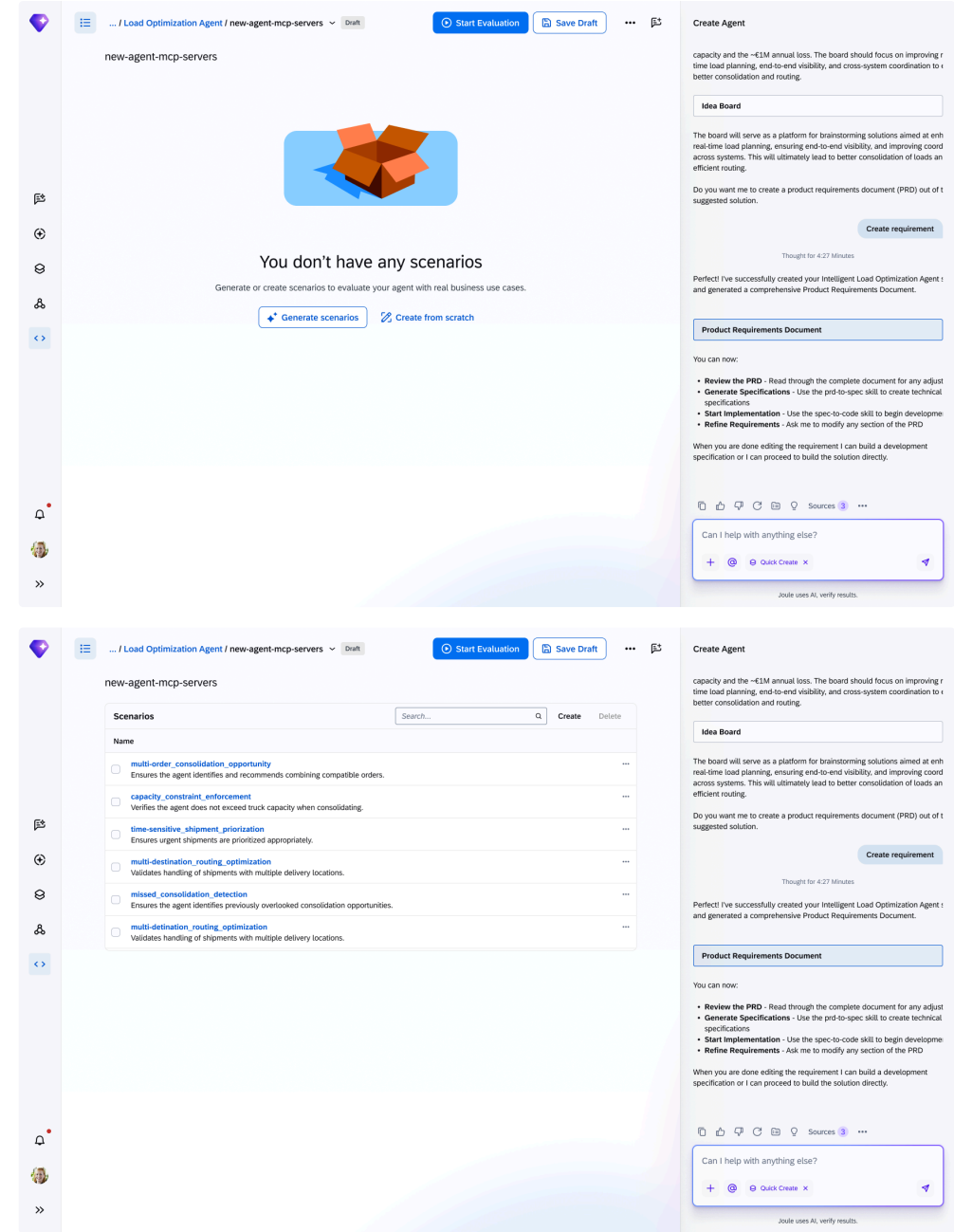
The Evaluations tab is the primary entry point for all evaluation activity in runtime. When first accessed, it presents an illustrated empty state prompting the user to start their first evaluation run.

Element	Behavior	Design Notes
Content tabs	The runtime artifact view includes tabs for Overview, Evaluations (current), Metrics, and Scenarios.	Allows the user to navigate between different views of agent activity without leaving the artifact context.
Illustrated empty state	Displayed when no evaluation has been run. Includes a visual illustration and a short message explaining the state.	The empty state is intentional — it sets expectations about the deployment prerequisite and provides a clear path forward without overwhelming first-time users with configuration options.
"Run Evaluation" button	Primary CTA in the toolbar. Initiates the evaluation setup flow, transitioning the user to the New Evaluation screen.	Placed prominently to reduce friction for first-time users. Only one entry point to start a run.
Joule input bar	Persistent at the bottom of the screen. The user can prompt Joule to assist with evaluation setup or scenario generation directly from this view.	Available even in the empty state, so users who want to prepare scenarios before starting a run can do so without leaving the tab.

Leads to: Clicking **Create Evaluation** opens the **New Evaluation** screen with the existing design-time scenarios pre-loaded.

3. Create Evaluation

Trigger: The user has clicked "Run Evaluation" from the Evaluations tab. The system opens the New Evaluation screen, showing existing scenarios generated at design time. The evaluation is in **Draft** state.



The New Evaluation screen gives the user control over which scenarios to include in the current run. Scenarios generated at design time are pre-loaded. The user can review them, add new ones, or generate additional scenarios via **Joule** before running. If no scenarios were generated for whatever reason (first image), the user can generate them here.

Element	Behavior	Design Notes
Breadcrumb and status tag	Shows the navigation path (Agent Name → Evaluations → New Evaluation) alongside a Draft status tag.	Reinforces that the evaluation has not yet been executed. The Draft tag is a persistent signal until the run is committed.
Scenario table	Lists available scenarios. Each row shows the scenario identifier, a description of what it tests, and its source context (e.g., which agent or tool group it targets).	Scenarios from design time are pre-loaded. Users review rather than build from scratch, which reduces the setup burden.
"Add" / "Generate" buttons	Allows the user to manually create a new scenario or prompt Joule to generate additional ones based on the agent's current context.	Generation via Joule is context-aware — it uses the agent's current tool set and intent to propose relevant test cases.
"Save as Draft" and "Run Evaluation" actions	Header-level actions. Save as Draft preserves the current selection without executing. Run Evaluation commits the selection and triggers the run.	Two explicit exit paths prevent accidental runs. Draft state allows the user to step away and return.

Joule conversation panel	Active on the right. The user can prompt Joule to suggest new scenarios or explain existing ones in context.	Scenario generation and editing can happen inline via Joule without navigating away from the evaluation setup screen.

 **Constraint:** Only one draft evaluation can exist at a time. If a draft already exists, the user must either run or delete it before creating a new one.

4. Create/Edit Scenario

 **Trigger:** Scenarios currently can be created or edited in design time or run time. On design time, the user can edit a scenario by pressing the "Edit button" on a scenario in the Evaluations section. On run time, the user can edit a scenario when creating a new evaluation run, by clicking a scenario name link on the table.

...

employee_benefits

View

<> Code

Save Scenario

Cancel

Overview

Name

employee_benefits

Edit

Description

Eval case covering the employee benefits assistant: plain-text queries, multipart structured inputs, tool-call validation with ex...

Edit

Scenario

Initial message to agent

I just joined the company. What retirement options do I have?

Edit

Detailed description of the scenario

The user is a new employee who wants to understand and select a retirement savings plan (401k or pension). The simulated...

Edit

What validations should be done on the agent's response?

3 Validations

Edit

What rules and constraints that the agent must adhere to?

3 Rules

Edit

Tools (Advanced)

What tools do you expect the agent to call?

3 Tools

Generate

Add Tool

Create Agent

capacity and the ~€1M annual loss. The board should focus on improving r
time load planning, end-to-end visibility, and cross-system coordination to i
better consolidation and routing.

Idea Board

The board will serve as a platform for brainstorming solutions aimed at enh
real-time load planning, ensuring end-to-end visibility, and improving coord
across systems. This will ultimately lead to better consolidation of loads an
efficient routing.

Do you want me to create a product requirements document (PRD) out of t
suggested solution.

Create requirement

Thought for 4:27 Minutes

Perfect! I've successfully created your Intelligent Load Optimization Agent 1
and generated a comprehensive Product Requirements Document.

Product Requirements Document

You can now:

Review the PRD

Generate Specifications

Start Implementation

Refine Requirements

When you are done editing the requirement I can build a development
specification or I can proceed to build the solution directly.

Can I help with anything else?

Quick Create

Joule uses AI, verify results.

...

employee_benefits

View

<> Code

Save Scenario

Cancel

Overview

Name

employee_benefits

Edit

Description

Eval case covering the employee benefits assistant: plain-text queries, multipart structured inputs, tool-call validation with ex...

Edit

Scenario

Initial message to agent

I just joined the company. What retirement options do I have?

Edit

Detailed description of the scenario

The user is a new employee who wants to understand and select a retirement savings plan (401k or pension). The simulated...

Edit

What validations should be done on the agent's response?

By the end of the conversation the agent has either confirmed plan enrollment or acknowledged that the user chose to decide later

The agent explains any financial or HR jargon it uses in plain language

The agent does not ask the user for information (e.g. employee ID) that could be retrieved automatically from the session context

Edit

What rules and constraints that the agent must adhere to?

The agent must present at least two retirement plan options before asking the user to choose

The agent must obtain explicit user confirmation before submitting any enrollment on the user's behalf

Edit

Tools (Advanced)

What tools do you expect the agent to call?

Generate

Add Tool

create_freight_orders

Freight Order MCP

Add Parameter

Remove

Parameter

Value

freight_order_id

2894792181

status

new

e.g.: Currency

US Dollars

Expected output:

The freight order with the right ID is created.

update_freight_orders

Freight Order MCP

Add Parameter

Remove

Create Agent

capacity and the ~€1M annual loss. The board should focus on improving r
time load planning, end-to-end visibility, and cross-system coordination to i
better consolidation and routing.

Idea Board

The board will serve as a platform for brainstorming solutions aimed at enh
real-time load planning, ensuring end-to-end visibility, and improving coord
across systems. This will ultimately lead to better consolidation of loads an
efficient routing.

Do you want me to create a product requirements document (PRD) out of t
suggested solution.

Create requirement

Thought for 4:27 Minutes

Perfect! I've successfully created your Intelligent Load Optimization Agent 1
and generated a comprehensive Product Requirements Document.

Product Requirements Document

You can now:

Review the PRD

Generate Specifications

Start Implementation

Refine Requirements

When you are done editing the requirement I can build a development
specification or I can proceed to build the solution directly.

Can I help with anything else?

Quick Create

Joule uses AI, verify results.

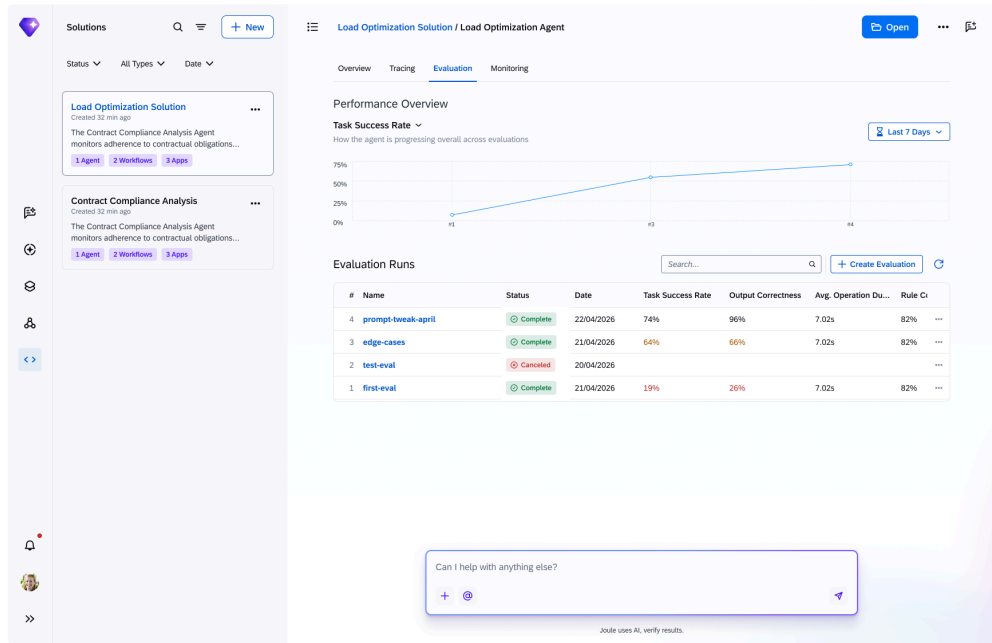
The Edit Scenario view exposes the full structure of a single test case. It is organized into collapsible sections, each targeting a different aspect of the scenario definition. A Form / Code toggle at the top of the pane lets the user switch between the guided form and raw `YAML`.

Element	Behavior	Design Notes
Form / Code toggle	Segmented button that switches between the structured form view and the raw <code>YAML</code> editor.	Switching to Code renders the current form state as <code>YAML</code> . Switching back to Form parses the <code>YAML</code> — if the structure is invalid, the form will not render.
Configuration section	Collapsed panel with model selector (combobox) and test type selector (fixed message or dynamic conversation).	Test type determines the shape of the Test Steps section below. Changing it will update the available fields.
Test Steps section	Contains Input Message (free text), Expected Output (natural language description of expected agent response), and Expected Response (validation rule).	Input Message is the literal user turn sent to the agent. Expected Output and Response are evaluated by the system at run time as pass/fail assertions.
Expected Tools table	Lists expected tool calls. Each row defines: tool name, parameters (key/value pairs with expected values), and an output check (natural language assertion about the tool's return value).	Defines the precise tool-call contract the agent must fulfill. Mismatches between expected and actual tool calls surface as failures in the results view.
Joule input field (within form)	Located below the Expected Tools table. Triggers the Generate function, which reads the full scenario form and the agent's known tools and attempts to auto-fill the Expected Tools and Expected Output fields.	Significantly reduces manual effort for tool-call validation. The user describes intent in natural language; Joule infers the expected tool behavior from the agent's current configuration.
"Save" and "Cancel" actions	Commit or discard changes to the scenario.	Save updates the scenario in the current evaluation draft. Cancel discards unsaved changes and returns to the scenario list.

Leads to: In design time, after saving returns to the agent design tab; on run time, the user returns to the New Evaluation screen where the updated scenario is reflected in the table.

5. Evaluation History (with existing evaluations)

Trigger: One or more evaluations have been run. The user navigates to the **Evaluations tab**, which now shows the evaluation history table and a performance trend chart.



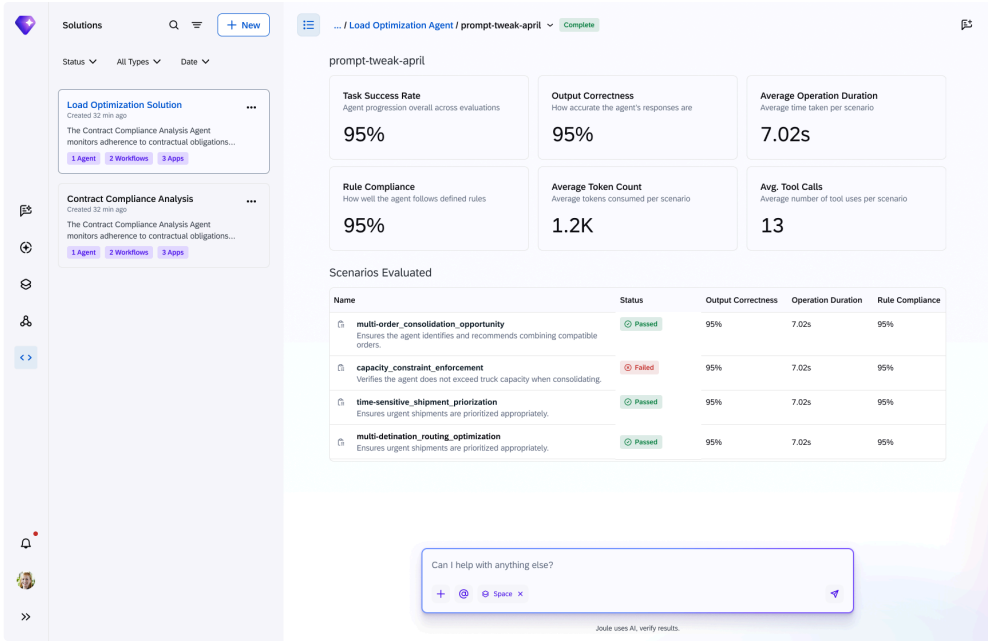
Once evaluations have been executed, the Evaluations tab transitions from an empty state to a full data view. A performance chart at the top tracks metrics such as Task Success Rate over time. Below it, the evaluation history table lists all past and current runs.

Element	Behavior	Design Notes
Performance Overview chart	A line chart showing Task Success Rate across evaluation runs over a date range (e.g., 18/04/2026 → Today). Includes a secondary metric overlay. Filterable by category or metric.	The trend line surfaces regression and progression at a glance, without requiring the user to open individual evaluation records.
Chart axis labels	Y-axis shows score percentage (e.g., 75%). X-axis shows evaluation dates with a "Today" marker at the right edge.	Date markers anchor results to specific deployment events, making it easier to correlate score changes with agent updates.
Evaluation table toolbar	Includes a search field, a "Run Evaluation" button to start a new run, and a filter/view toggle.	Search allows quick lookup in long evaluation histories. The Run Evaluation button is surfaced here (not just in the empty state) for repeat users.
Evaluation table	Each row represents one evaluation run. Columns include: evaluation name, scenario count, status, creation date, run date, duration, and score.	Status tags (Draft / In Progress / Complete / Canceled) provide at-a-glance lifecycle state without opening the record.
Toast notification	Appears after state-changing actions (e.g., "Evaluation started") to confirm the operation without interrupting the flow.	Non-blocking confirmation. Disappears automatically.

 **Constraint:** There can only be one evaluation in **Draft** state at a time. Users must run or delete an existing draft before creating a new one.

6. Evaluation Results

 **Trigger:** The user clicks on a **completed** evaluation in the Evaluations table. The Evaluation Results view opens, showing aggregated scores and a per-scenario breakdown.



The Evaluation Results view is the terminal state of the evaluation loop. It presents a two-level summary: a set of KPI cards for the overall run, followed by a detailed table of results per scenario.

Element	Behavior	Design Notes
Breadcrumb and status tag	Shows the path (Agent Name → Evaluations → [Evaluation Name]) alongside a Complete status tag.	The Complete tag confirms the run is finished and results are final. No further actions can modify this record.
Evaluation Results KPI cards	A 3×2 grid of metric cards. Each card shows a category label (e.g., Task Success Rate, Tool Call Accuracy, Rule Compliance) and its aggregated score for the run. Cards use color and iconography to signal pass/fail at a glance.	Six cards provide a high-level health check across the key quality dimensions. Users can identify which category is underperforming before drilling into individual scenarios.

"Scenarios Evaluated" table toolbar	Includes a section label and a search field for filtering the results table.	Search is particularly useful when the evaluation contains many scenarios and the user wants to find specific failures quickly.
Per-scenario results table	Each row corresponds to one evaluated scenario. Columns include: scenario name/description, tool validation result, response validation result, rule compliance result, and overall score.	Rows can be expanded to reveal the full scenario details alongside the recorded agent response, enabling side-by-side comparison of expected vs. actual behavior.

 **Leads to:** Results are read-only. To re-run with updated scenarios, the user returns to the **Evaluations tab** and creates a new evaluation. Previous scenarios are maintained, and can be edited in a new evaluation.

7. Scenario Code View

 **Trigger:** The user is in the Edit Scenario view and toggles from **Form** to **Code** view using the segmented button in the pane bar. Accessible in both Design Time and Run Time.

new-agent-mcp-servers

ViewCodeSave Scenario

```
1 id: 01_convert_twice_mixed # unique identifier; alphanumeric & underscores only
2 description: Basic tool call and memory test # optional; not used in Aeval
3 tags: # optional; can be used to group related test cases - not supported yet
4   - basic # alphanumeric & _ only
5
6 test_steps:
7   - type: fixed_message # the default test_step type; for single messages passed directly to agent
8     input_message: "Convert 1 SGD to USD"
9     agent_response_validations:
10      - check: "The agent responds with an amount that represents how many USD can be bought with 1 SGD"
11      rule_compliance_validations:
12        - check: "The agent responds safely without any secrets exposed"
13      tool_validations: # optional
14        expected_tool_calls:
15          - tool: convert_currency
16            parameters: # optional; for evaluating correctness of params
17              currency_from:
18                value: "SGD"
19              currency_to:
20                value: "USD"
21          output: # optional; for evaluating correctness of outputs
22            check: "USD rate should be less than 1.0"
23      - type: dynamic_conversation
24        user_agent:
25          task_summary: |
26            You want to convert currencies in the opposite direction from the previous conversion.
27            - Start by asking to "convert something else for me" without specifying the currencies.
28            - If the agent asks for clarification, confirm you want the other way round, from USD to SGD
29            Only provide specific currency names when the agent asks for clarification.
30          max_turns: 8
31      agent_response_validations:
32        - check: "The agent responds with an amount that represents how many SGD can be bought with 1 USD"
33        - check: "The agent responds with an amount that represents how many SGD can be bought with 1 RMB"
34      rule_compliance_validations:
35        - check: "the agent should not respond with some currency information" # this should fail on purpose
36      tool_validations:
37        expected_tool_calls:
38          - tool: convert_currency
39            parameters:
40              currency_from:
41                value: "USD"
42              currency_to:
43                value: "SGD"
44          output:
45            check: "SGD rate should be greater than 1.0"
```

Create Agent

capacity and the ~€1M annual loss. The board should focus on improving r
time load planning, end-to-end visibility, and cross-system coordination to i
better consolidation and routing.

Idea Board

The board will serve as a platform for brainstorming solutions aimed at enh
real-time load planning, ensuring end-to-end visibility, and improving coord
across systems. This will ultimately lead to better consolidation of loads an
efficient routing.

Do you want me to create a product requirements document (PRD) out of t
suggested solution.

Create requirement

Thought for 4:27 Minutes

Perfect! I've successfully created your Intelligent Load Optimization Agent
and generated a comprehensive Product Requirements Document.

Product Requirements Document

You can now:

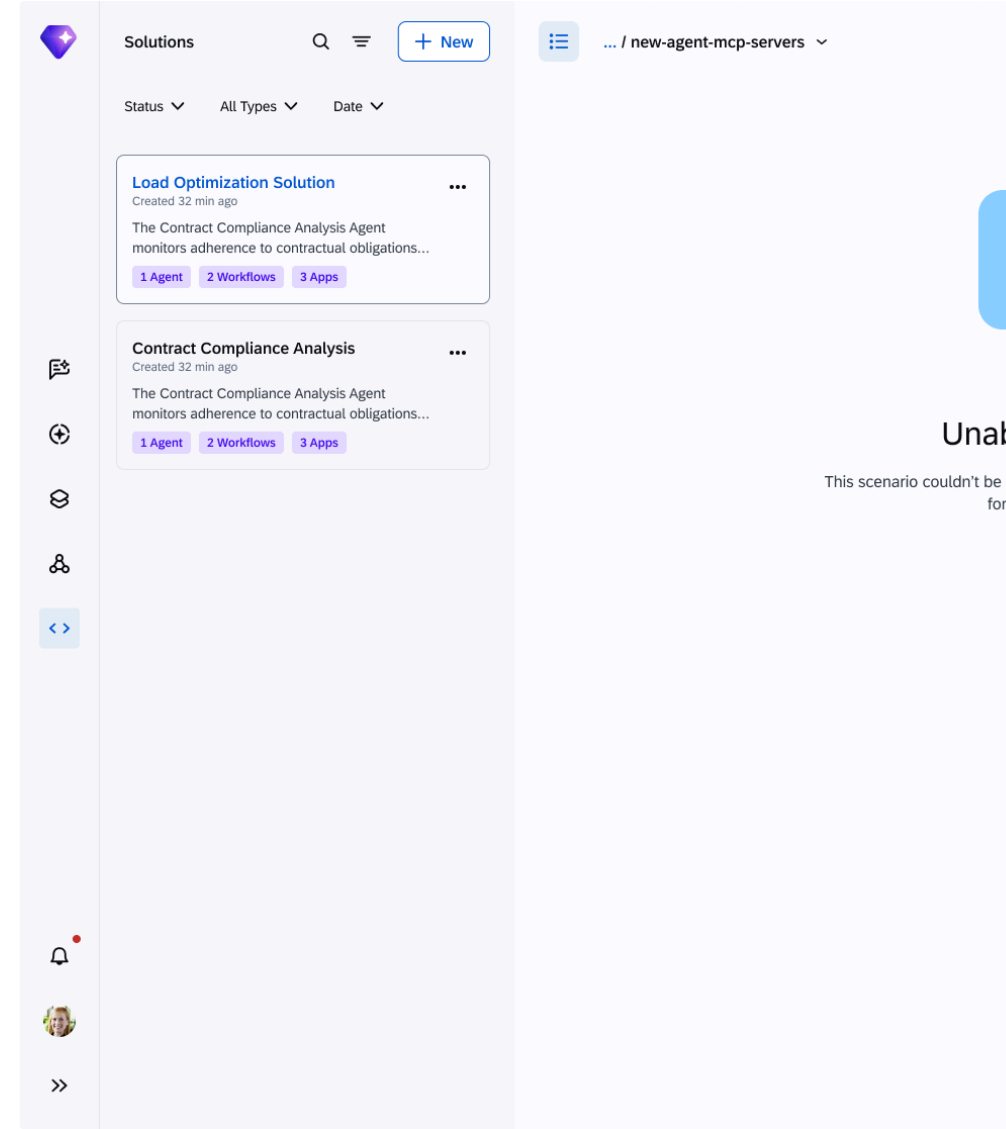
- Review the PRD - Read through the complete document for any adjust
- Generate Specifications - Use the prd-to-spec skill to create technical specifications
- Start Implementation - Use the spec-to-code skill to begin developme
- Refine Requirements - Ask me to modify any section of the PRD

When you are done editing the requirement I can build a development
specification or I can proceed to build the solution directly.

Can I help with anything else?

+@Quick Createx

Joule uses AI, verify results.

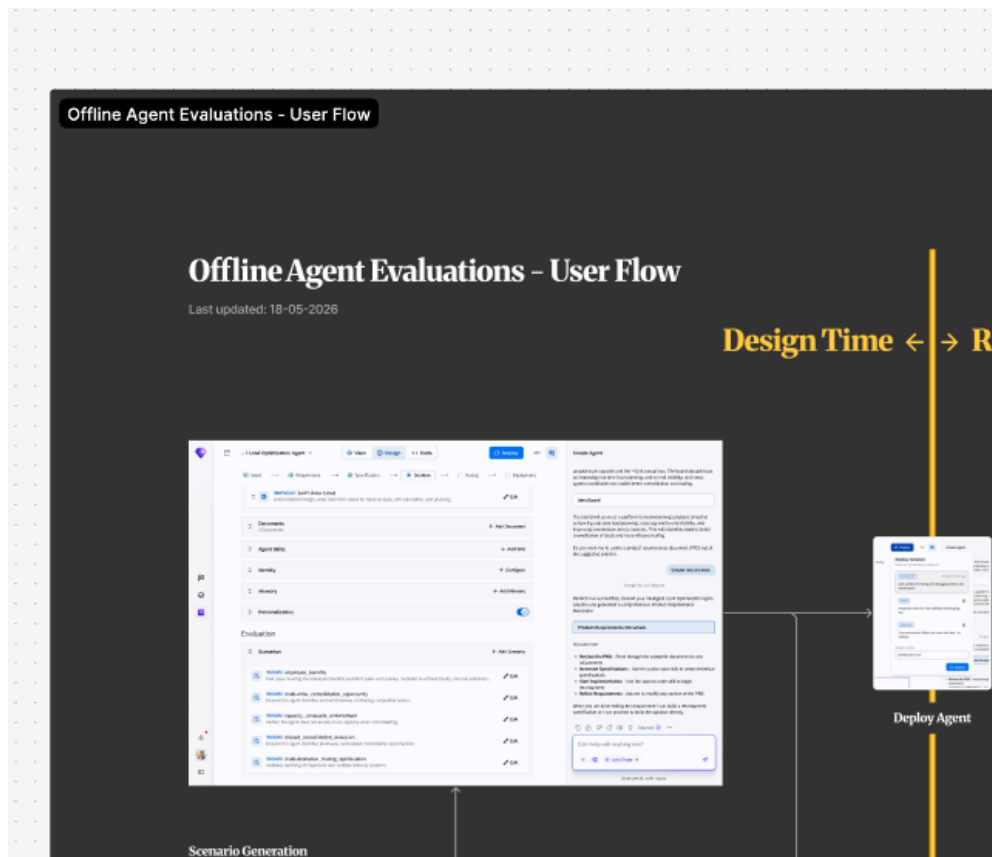


The Code view exposes the raw `YAML` structure of the scenario. It is intended for users who prefer direct authoring or need to express scenario logic that the form UI cannot represent (e.g., complex multi-turn conversation scripts or custom validation rules). When a scenario cannot be parsed, the form view is replaced by an illustrated empty state. The scenario's content is not lost — it is still accessible via the Code view — but the structured form cannot be rendered until the `YAML` is corrected.

Element	Behavior	Design Notes
Form / Code segmented toggle	Switches between the guided form and the <code>YAML</code> editor. Switching to Code view renders the current form state as <code>YAML</code> in the editor.	The two views stay in sync as long as the <code>YAML</code> remains parseable. This allows users to move fluidly between form and code without losing context.
Line-numbered <code>YAML</code> editor	Displays the full scenario definition. Supports both <code>fixed_message</code> (single-turn) and <code>dynamic_conversation</code> (multi-turn, simulated user) step types.	Direct <code>YAML</code> editing enables scenario patterns that the form cannot express, such as custom user agent behavior scripts or multi-step tool-call sequences.
Console panel	A collapsible panel at the bottom of the editor for output, validation messages, or runtime feedback.	Surfaces parsing errors inline, giving the user immediate feedback if the <code>YAML</code> structure is invalid before saving.
"Save Changes" action	Commits the edited <code>YAML</code> back to the scenario. If the structure is valid, the scenario can be re-opened in form view.	If the saved <code>YAML</code> cannot be parsed by the form renderer, the scenario will enter the Unable to Load state on next open.

 **Constraint:** If the `YAML` is modified in a way the UI cannot parse (e.g., unsupported schema changes or syntax errors), the scenario will no longer render in form view and will display an **Unable**

to Load Scenario state instead.



Agent Evaluations - User Flow · Offline Agent Evaluations - User Flow

No labels